

An Optimized GPU-based Acceleration of CRYSTALS-Dilithium

Zhao Chen*, Weimin He*, Jiafei Wu[†], Jingjie Liu, Boqin Xu, Xiaoning Bian, and Zhe Liu[†]

Research Center for Data Hub and Security, Zhejiang Lab, Hangzhou, China

Email: {chenzhao, heweiemin, wujiafei, liujingjie, xuboqin, bian, zhe.liu}@zhejianglab.org

Abstract—CRYSTALS-Dilithium has recently been selected as one of the next generation post-quantum signature algorithm standards. However, due to the extensive volume of data elements and the high complexity of operations, post-quantum cryptographic algorithms commonly face significant performance challenges, for which GPU acceleration has proven to be an effective hardware solution. This paper presents an optimized GPU-based implementation of Dilithium, accelerating the algorithm across three dimensions: inter-thread, intra-thread, and inter-block. Specifically, we propose shared memory and loop unrolling techniques to optimize the most time consuming number theoretic transform operation. Meanwhile, we employ a PTX assembly implementation of Montgomery modular multiplication for frequent dot product computations. Experimental results indicate that, compared to the NIST standard implementation, our implementations of Dilithium across all three parameter sets achieve speedups of up to 143x for Gen, 255x for Sign, and 152x for Verify on the GPU 4090. On the GPU A100, the speedups reach 114x for Gen, 158x for Sign, and 124x for Verify.

Index Terms—Dilithium, post-quantum cryptography, GPU, number theoretic transform

I. INTRODUCTION

Digital signature is a cryptographic primitive that ensures data integrity and verifies identities in various applications, including e-commerce, email, and blockchain. However, the advent of quantum computing technologies poses a significant threat to the security of traditional digital signature algorithms, such as the Rivest-Shamir-Adleman (RSA) signature [1] and the Elliptic Curve Digital Signature Algorithm (ECDSA) [2]. These algorithms rely on discrete logarithm and large integer prime factorization problems, which can be efficiently solved by Shor's algorithm [3]. Therefore, developing post-quantum cryptography (PQC) to withstand quantum attacks is essential for ensuring future data security.

To address quantum threats, in 2016, the National Institute of Standards and Technology (NIST) launched a global competition to standardize PQC algorithms and Dilithium [4] has been selected as one of the final signature standards. However, like other PQC algorithms, Dilithium faces performance challenges due to its computational complexity, which can increase processing time and resource consumption. Graphical Processing Unit (GPU) acceleration is frequently utilized to address these challenges, leveraging the parallel processing capabilities of GPUs to optimize cryptographic computations. So far, there are many studies focus on the

GPU-based acceleration in Dilithium [5]–[8]. However, several challenges remain, including optimizing memory usage, balancing workloads among GPU cores, and reducing latency.

In this paper, we propose an optimized GPU-based acceleration of Dilithium from three dimensions: inter-thread, intra-thread, and inter-block, where inter-thread optimization is the focus of our research to improve the overall performance by enhancing collaboration methods between threads. In addition, for the most time consuming Number Theoretic Transform (NTT) operation, we applied optimization techniques such as shared memory and loop unrolling. Moreover, we employ a PTX assembly implementation of Montgomery modular multiplication for frequent dot product computations. Experimental results demonstrate that our optimization strategy significantly improves the performance of Dilithium.

The remainder of this paper is organized as follows. Section II reviews related works and introduces Dilithium algorithm and GPUs. The proposed method is elaborated in Section III, while the experimental results are presented in Section IV. Our final remarks are summarized in Section V.

II. PRELIMINARIES

A. Related works

The optimized implementations of the Dilithium algorithm have been studied across various platforms, including FPGA [9]–[11], ARM [12], [13], RISC [14], and AVX [15]. A comparison of cycle counts for these implementations is summarized in Table I. Each platform has made advancements due to its unique capabilities. FPGAs [9] utilize parallelization to reduce cycle counts and increase operational frequencies. Cortex-M3 and Cortex-M4 processors [12] enhance polynomial multiplication through NTT and its inverse (INTT). Cortex-A72 and M1 [13] processors efficiently combine Montgomery multiplication with Barrett reduction using Neon vector instructions. RISC-V [14] employs a hardware/software co-design approach with adaptable hardware accelerators for Dilithium schemes. AVX2 and AVX-512 [15] instructions also improve the efficiency of NTT operations and the processing of polynomial coefficients, leading to better performance.

Despite these advancements, research on GPU-based implementations of Dilithium remains relatively limited. GPUs possess substantial parallel processing power, providing the potential to accelerate the computationally intensive operations associated with Dilithium. Some studies [5]–[8] have explored this area, where Seo et al. [8] introduced several optimization

* Both authors contributed equally to this research.

[†] Jiafei Wu and Zhe Liu are the corresponding authors.

TABLE I
CYCLE COUNTS WITH RELATED WORKS FOR DILITHIUM3

Ref	Platform	Gen	Sign	Verify
[9]	FPGA Virtex 7	18193	21033	15032
[12]	ARM Cortex-M4	2013	6053	6053
[13]	Apple M1	152435	365248	104821
[14]	RISC-V PULPino	1067824	3253378	1126938
[15]	AVX-512	43102	146636	146636

techniques, including a warp divergence reduction method based on dummy operations, a parallel implementation of NTT-based polynomial multiplication, and an optimized rejection sampling process utilizing a rejection sequence table, which improve the efficiency of Sign and Verify processes.

B. CRYSTALS-Dilithium

Dilithium is a lattice-based digital signature scheme that based on the Module Learning With Errors [4] problem. It emerged as a leading candidate in the Round-3 of the NIST PQC competition and advanced to the Draft Federal Information Processing Standard (FIPS) 204 [16] stage in 2023. Following a rigorous review and testing process, NIST has recently published the final FIPS 204 standard [17].

Dilithium consists of three polynomial-time algorithms: Gen, Sign, and Verify, see the NIST supporting documentation for details [18]. The algorithm Gen outputs Public Key (pk) and Secret Key (sk). In the signature phase, the message is hashed and combined with sk to produce the signature. The verification phase employs pk to validate the authenticity of the signature. For these three algorithms, the performance bottlenecks mainly occurs in polynomial multiplication, particularly during the NTT/INTT operations. To address these computationally intensive operations, GPU acceleration offers a novel solution. By leveraging the parallel processing capabilities of GPUs, significant improvements in efficiency can be achieved, thereby enhancing the performance of Dilithium. This advancement not only reduces the latency associated with polynomial operations but also highlights the potential of hardware acceleration in optimizing PQC algorithms.

C. GPU architecture and CUDA programming

GPUs [19], [20] are meticulously crafted for high-throughput parallel processing, distinguishing them from traditional Central Processing Units (CPUs) that are optimized for sequential task execution. A typical GPU is composed of thousands of smaller, efficient cores capable of executing numerous threads simultaneously, rendering them particularly suitable for computation-intensive tasks such as graphics rendering, scientific simulations, and cryptographic operations.

Compute Unified Device Architecture (CUDA) [21], [22] is a powerful parallel computing platform and programming model that enables developers to leverage the computational capabilities of NVIDIA GPUs. Within the CUDA programming model, the system is divided into the host (CPU) and the device (GPU). The host runs the main program and manages the overall execution flow, while the device executes parallel kernels—functions performed on the GPU. A critical aspect of

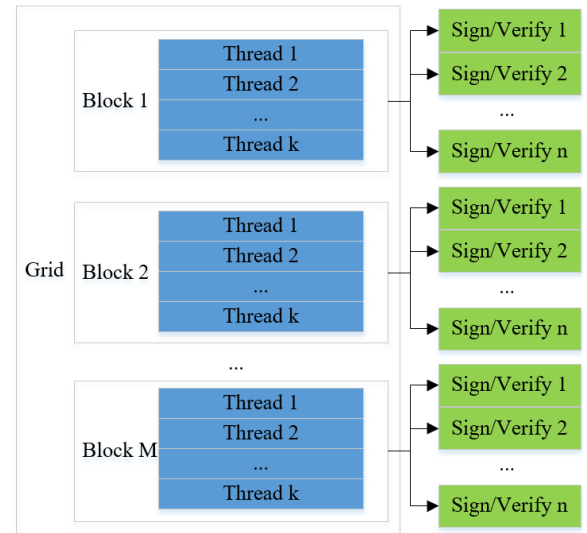


Fig. 1. Parallel design in GPU environment.

CUDA programming is the data transfer between the host and the device. The host initializes and transfers data to the device memory, where the GPU conducts the necessary computations. Upon completion, the results are sent back to the host for further processing or storage. This division of labor allows CUDA to fully utilize the immense parallelism inherent in GPUs, significantly accelerating computational tasks.

III. METHOD

We deploy multiple blocks on the GPU, each block using k threads, and each group of k threads processing multiple Sign/Verify procedures, see Fig. 1. According to this model, we employ three types of optimization methods: inter-thread optimization within a single block, intra-thread optimization, and optimization across multiple blocks. Among these, inter-thread optimization within a single block is our primary focus, aiming to enhance the collaboration methods between threads to improve overall performance. Intra-thread optimization seeks to increase the execution speed of individual threads. Inter-block optimization involves adjusting the number of parallel blocks and finding suitable parameters to ensure that the GPU operates at full capacity while also preventing performance degradation due to block scheduling.

A. Inter-thread optimization within a single block

The NTT is the most time-consuming operation in Dilithium implementation, thus we focus on optimizing the NTT. Given that Dilithium processes only the 256-point NTT, our analysis is restricted to this specific case.

In the unoptimized implementation version, we first use 128 threads per block to process a single NTT operation, see Fig. 2. At each level, each thread processes only 2 points. Since the point-pairs are different at each level, the threads first read data from global memory and writes the data back to global memory after processing, resulting in a significant amount of data that needs to be synchronized. Furthermore, to ensure effective data synchronization, thread synchronization within the block is required after writing data at each level. Note

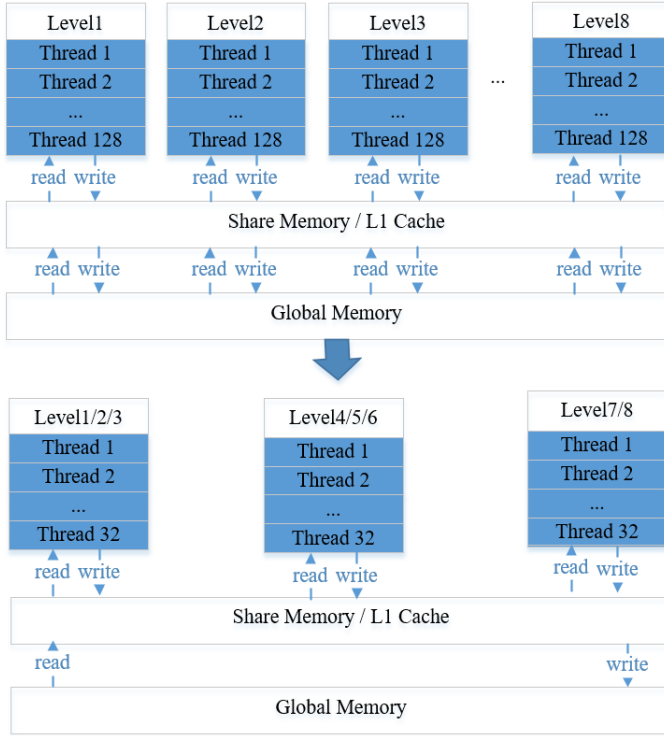


Fig. 2. NTT implementation on GPU. Memory read/write is a time-consuming operation. We focus on optimizing memory read/write.

that the data in global memory is synchronized through the L1 Cache, if the L1 Cache remains valid, synchronization can occur only within the L1 Cache without needing to update global memory each time. However, during the actual implementation, it is not guaranteed that the L1 Cache will remain valid, which affects data synchronization performance.

Since CUDA provides a shared memory mechanism that does not rely on the L1 Cache for automatic data synchronization. In the optimized implementation version, we can preallocate shared memory for 256 integers and first copy the data from global memory to shared memory. During the intermediate eight levels of computation, data is synchronized through shared memory, and finally the data is copied from shared memory back to global memory.

Additionally, to further reduce the number of data synchronization, we considered another approach by reducing the 128 threads to 32 threads. In this case, each thread in each level processes 8 points. By carefully designing the organization of these points, we can process these 8 points at levels 1/2/3 without requiring inter-thread data synchronization. Similarly, levels 4/5/6 do not require data synchronization, nor do levels 7 and 8. Consequently, only 3 inter-thread data synchronization is required. Furthermore, the 32 threads conform to the thread warp scheduling mechanism of GPU, allowing for warp synchronization instead of block-level thread synchronization.

B. Intra-thread optimization

Within a single thread, we enhance algorithm performance by employing PTX assembly and loop unrolling techniques. Modular arithmetic in Montgomery form is a widely used

TABLE II
THROUGHPUT OF C AND AVX2 IMPLEMENTATIONS FOR NTT OPERATIONS ON CPU, OUR IMPLEMENTATIONS ON GPU A100 AND GPU RTX4090. THE METRIC IS THE OPERATIONS PER MILLISECOND.

Ref	AVX2	A100 ntt128	A100 ntt32	4090 ntt128	4090 ntt32
522	3582	446332	448652	500218	482659
(-)	(-)	(855×)	(859×)	(958×)	(924×)

method for improving the efficiency of modular multiplication. During the butterfly processing of NTT, Montgomery modular multiplication is frequently utilized. Therefore, we replace the C code implementation of Montgomery modular multiplication with PTX assembly to maximize instruction performance.

Additionally, within the NTT processing function, there are small loops with lengths of 8 and 4. To reduce the number of instructions executed, we employ loop unrolling techniques to remove instructions related to the calculation and checking of loop variables, thereby enhancing intra-thread execution performance. CUDA provides preprocessor directives that allow the nvcc compiler to automatically perform loop unrolling optimizations when compiling C code into machine code.

C. Inter-block optimization

Currently, there is no additional optimization has been implemented across multiple blocks, the only adjustment made is the number of parallel blocks. By identifying appropriate parameters, it is possible to ensure that the GPU operates at full capacity, thereby preventing performance degradation that may arise from block scheduling.

IV. EXPERIMENTAL RESULTS

We compile the C/C++ code using g++ 11.4.0 and the GPU implementations with CUDA 12.4 on an Arch Linux system with kernel 5.10. Our optimized implementations were deployed and tested on two GPUs: the NVIDIA Tesla A100 80G and the NVIDIA GeForce RTX4090.

The CPU baseline is the NIST standard reference C implementation [23], the performance of which is obtained on an Intel(R) Core(TM) i9-13900K CPU with 24 cores. To evaluate the three procedures of Dilithium, we measure performance in terms of operations per second (OP/s). The reported performance results are the medians of 10 executions, where each execution batches multiple tasks.

Since we mainly optimize the most time consuming NTT operations, we design two test sets. The first test set presents the performance of individual NTT operations in Dilithium implementation. The second test set presents the performance of Dilithium, including Gen, Sign and Verify procedures.

A. Number-Theoretic Transform

We design three NTT versions: (1) ntt128 uses 128 threads for NTT calculation; (2) ntt128_sm uses shared memory to share the calculation results of 128 threads on the basis of ntt128; (3) ntt32 uses 32 threads for NTT calculation. Note that although ntt32 version has a low thread parallelism, it reduces the number of data synchronization times, and it also uses shared memory to synchronize data. We also design different

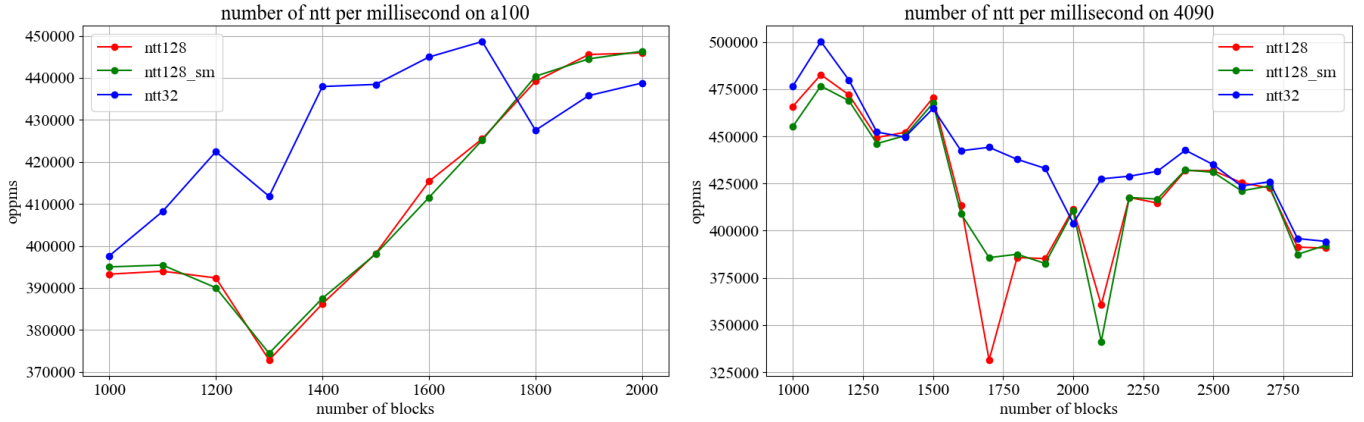


Fig. 3. NTT implementations on a100 and 4090, the metric is the operations per millisecond.

TABLE III

THROUGHPUT OF C AND AVX2 IMPLEMENTATIONS FOR DILITHIUM ON CPU, OUR OPTIMIZED IMPLEMENTATIONS ON GPU A100 AND GPU RTX4090, AND RELATED WORKS ON NVIDIA JETSON AGX XAVIER [8], ARM CORTX-A72 [13], AND FPGA [11]. THE METRIC IS THE OPERATIONS PER SECOND (OP/S). THE SPEEDUPS REFER TO THE COMPARISON WITH CPU REFERENCE IMPLEMENTATION.

		Dilithium2			Dilithium3			Dilithium5		
		Gen	Sign	Verify	Gen	Sign	Verify	Gen	Sign	Verify
CPU	Ref [23]	12679	2893	11636	6792	1825	7455	4650	1477	4515
	AVX2 [6]	67188	23678	67084	41510	15415	42210	26421	12728	26839
Our Implementations	RTX 4090	1626223	644632	1671963	975188	456003	1072122	661193	377480	687617
		(128×)	(222×)	(143×)	(143×)	(249×)	(143×)	(142×)	(255×)	(152×)
	A100	1368549	422011	1449826	777750	289507	885427	494006	216128	529930
		(107×)	(145×)	(124×)	(114×)	(158×)	(118×)	(106×)	(146×)	(117×)
Related Works	[8]	84993	33965	67738	51099	14875	44502	31800	20396	27511
	[13]	5561	2310	5498	2908	1377	3352	1916	1044	1961
	[11]	23217	3448	21904	16555	2167	15671	11051	1977	10716

blocks and compare the number of NTT operations that can be performed per millisecond.

In Fig. 3, we compare the performance of individual NTT operations on A100 with the number of parallel blocks between 1000 and 2000. When blocks is less than 1750, ntt32 is better than ntt128. When blocks is greater than 1750, ntt128 is better than ntt32. In this case, ntt32 will use more register resources, but when the register resources are insufficient, it will exchange register data through the global memory, resulting in performance degradation. Meanwhile, we compare the performance of NTT operations on RTX4090 with the number of parallel blocks between 1000 and 3000. In this range, ntt32 is better than ntt128. When parallel blocks are 1100, ntt32, ntt128 and ntt128_sm all have the maximum processing capacity.

Meanwhile, we also implement the standard reference C version and AVX2 version for NTT operations on the CPU (i9-13900k), see Table II. From Fig. 3, our optimal implementations demonstrates throughputs of 448k and 500k operations per millisecond on GPU A100 and GPU RTX4090, respectively, corresponding to 859× and 958× speedups over a single-thread CPU implementation.

B. Performance on Different GPUs for Dilithium

The Table III shows the performance of our implementations and the comparisons. We implement the standard reference

C version and AVX2 version for dilithium on the CPU (i9-13900k). For the related work [8], [11], [13], we use the reported results from their papers. Compared to the CPU baseline, on the GPU RTX4090, our implementations achieve 128×-143× improvement for Gen, 222×-255× improvement for Sign, and 143×-152× improvement for Verify. On the GPU A100, our implementations achieve 106×-114× improvement for Gen, 145×-158× improvement for Sign, and 117×-124× improvement for Verify.

V. CONCLUSION

This paper presents an optimized GPU-based acceleration of Dilithium from three dimensions: inter-thread, intra-thread and inter-block optimization. For inter-thread optimization, we adopt a fine-grained NTT approach combined with shared memory to reduce data copy latency. For intra-thread optimization, we use PTX assembly and loop unrolling techniques to enhance modular multiplication efficiency. For inter-block optimization, we adjust the number of parallel blocks and apply optimal parameters to maximize parallel efficiency. Experimental results demonstrate that our optimization strategy significantly improves the performance of Dilithium. Thus, this effort contributes to the ongoing development and application of PQC algorithms in the future.

REFERENCES

- [1] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [2] D. Johnson, A. Menezes, and S. Vanstone, "The elliptic curve digital signature algorithm (ecdsa)," *International journal of information security*, vol. 1, pp. 36–63, 2001.
- [3] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.
- [4] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "Crystals-dilithium: A lattice-based digital signature scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 238–268, 2018.
- [5] X. Zhao, B. Wang, Z. Zhao, Q. Qu, and L. Wang, "Highly efficient parallel design of dilithium on gpus," 2022.
- [6] S. Shen, H. Yang, W. Dai, Z. Liu, and Y. Zhao, "High-throughput GPU implementation of dilithium post-quantum digital signature," *CoRR*, vol. abs/2211.12265, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2211.12265>
- [7] S. Shen, H. Yang, W. Li, and Y. Zhao, "cuml-dsa: Optimized signing procedure and server-oriented gpu design for ml-dsa," *Cryptology ePrint Archive*, 2023.
- [8] S. C. Seo and S. An, "Parallel implementation of crystals-dilithium for effective signing and verification in autonomous driving environment," *ICT Express*, vol. 9, no. 1, pp. 100–105, 2023.
- [9] S. Ricci, L. Malina, P. Jedlicka, D. Smékal, J. Hajny, P. Cibik, P. Dzurenda, and P. Dobias, "Implementing crystals-dilithium signature scheme on fpgas," in *Proceedings of the 16th International Conference on Availability, Reliability and Security*, 2021, pp. 1–11.
- [10] L. Beckwith, D. T. Nguyen, and K. Gaj, "High-performance hardware implementation of crystals-dilithium," in *2021 International Conference on Field-Programmable Technology (ICFPT)*. IEEE, 2021, pp. 1–10.
- [11] C. Zhao, N. Zhang, H. Wang, B. Yang, W. Zhu, Z. Li, M. Zhu, S. Yin, S. Wei, and L. Liu, "A compact and high-performance hardware architecture for crystals-dilithium," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 270–295, 2022.
- [12] D. O. Greconici, M. J. Kannwischer, and A. Sprenkels, "Compact dilithium implementations on cortex-m3 and cortex-m4," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 1–24, 2021.
- [13] H. Becker, V. Hwang, M. J. Kannwischer, B.-Y. Yang, and S.-Y. Yang, "Neon NTT: Faster Dilithium, Kyber, and Saber on Cortex-A72 and Apple M1," *Cryptology ePrint Archive*, 2021.
- [14] P. Karl, J. Schupp, T. Fritzmann, and G. Sigl, "Post-quantum signatures on risc-v with hardware acceleration," *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 2, pp. 1–23, 2024.
- [15] R. Xu, D. He, M. Luo, C. Peng, and X. Zeng, "Optimizing dilithium implementation with avx2/-512," *ACM Transactions on Embedded Computing Systems*, 2024.
- [16] National Institute of Standards and Technology (NIST), "Draft federal information processing standard (fips) 204: Crystals-dilithium," National Institute of Standards and Technology, Tech. Rep., 2023, available at <https://csrc.nist.gov/publications/detail/fips/204/draft>.
- [17] NIST, "Module-lattice-based digital signature standard," <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf>, accessed: 2024-08-22.
- [18] S. Bai, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "Crystals-dilithium algorithm specifications and supporting documentation (version 3.1)," 2021. [Online]. Available: <https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf>
- [19] J. D. Owens, M. Houston, D. Luebke, S. Green, J. E. Stone, and J. C. Phillips, "Gpu computing," *Proceedings of the IEEE*, vol. 96, no. 5, pp. 879–899, 2008.
- [20] J. Nickolls and W. J. Dally, "The gpu computing era," *IEEE micro*, vol. 30, no. 2, pp. 56–69, 2010.
- [21] S. Cook, *CUDA programming: a developer's guide to parallel computing with GPUs*. Newnes, 2012.
- [22] D. Guide, "Cuda c programming guide," *NVIDIA, July*, vol. 29, p. 31, 2013.
- [23] NIST, "Crystals-dilithium," <https://github.com/pq-crystals/dilithium/>.